AMD Artix-7 FPGA 仿视网膜事件驱动视觉系统

2026 全国大学生嵌入式芯片与系统设计竞赛 · FPGA 创新设计赛道

摘　要

传统相机采集视频时，即使画面没有变化，也会不断传输完整图像，产生大量重复数据，带来巨大的带宽消耗和后端处理压力。当被监控对象在时间上高度稀疏、变化只发生在少数时刻和少数空间位置时，用传统相机加高性能后端视频分析模块处理，大量算力会浪费在无效数据的处理与通信上，既抬高带宽需求，也放大系统总体复杂度。

针对这一问题，本文采用 OV5640 普通相机和 AMD Artix-7 FPGA，实现了一套高性能、低带宽需求的事件视觉系统。系统包括以 AMD Artix-7 FPGA 为核心的处理单元、OV5640 采集器、千兆网口与 UART 串口通信信道的事件采集前端，以及基于 Node.js + TypeScript 开发的上位机后端，通信协议支持全双工与热插拔。系统自动比较前后两帧画面，只记录变亮或变暗的位置，同时保留原始视频；原画流与事件流既可显示在板上 LCD，也可同步发送到上位机呈现。此外，我们基于变化事件流设计、训练并部署了一个轻量级板上 CNN 用于手势识别，识别结果可动态配置以控制上位机行为。

为在有限课余时间内完成开发，我们利用 Agent 工具搭建了集成仿真、综合、下载的全自动工作流，配合提炼的通用 FPGA 开发 Skill，把敏捷开发模式引入了开发流程，在有限时间内实现了较高的作品完成度。

系统运行期间，每隔约 37.5 ms 采集并处理一帧数据流，与上位机握手并通过可用信道交换数据包，每 100 ms 由 CNN 进行一次手势识别。网口支持千兆原始画面流与纯事件流的多模切换，UART 最高支持 921600 波特率的抽样事件流回传。板上 CNN 手势识别在动作段召回约 80%。实测在实验室常态场景下，全量事件数据量平均约为原始图像流的 1/171（≈ 115 kB/s）；串口抽样模式下再按 1/32 抽点，数据量可压缩到原始图像流的约 1/6000（≈ 3.4 kB/s）。该方案适合厂区周界监控、设备运行状态视觉监控、交通事件检测等更关注“何时、何地发生了变化”而不需要完整画面纹理的应用场景。

目前仓库已在 GitHub 上开源，

地址：https://github.com/p1gGod/RetinaEvent_FPGA[9]。

关键词：事件视觉；边缘计算；FPGA；轻量级卷积神经网络；手势识别；低带宽传输；

目　录

摘　要 1

第一部分　作品概述 4

1.1 功能与特性 4

1.2 应用领域 5

1.3 主要技术特点 5

1.4 主要性能指标 6

1.5 主要创新点 6

1.6 设计流程 7

第二部分　系统组成及功能说明 8

2.1 整体介绍 8

2.2 硬件系统介绍 9

2.2.1 硬件整体介绍 9

2.2.2 机械设计介绍 10

2.2.3 电路各模块介绍 10

2.3 软件系统介绍 11

2.3.1 软件整体介绍 11

2.3.2 软件各模块介绍 12

第三部分　完成情况及性能参数 19

3.1 整体介绍 19

3.2 工程成果 19

3.2.1 机械成果 19

3.2.2 电路成果 20

3.2.3 软件成果 20

3.3 特性成果 26

3.4　测试与验证方法 27

3.4.1　事件流与码率 27

3.4.2　端到端时延 28

3.4.3　动作识别评测 28

3.5　联机、操控与回传 30

3.5.1　热插拔与全双工 30

3.5.2　板上操控按键 31

第四部分　总结 34

4.1 可扩展之处 34

4.2 心得体会 34

第五部分　参考文献 37

第一部分　作品概述

1.1 功能与特性

本设计把 OV5640 的 800×480 帧流在板内转换成 400×240 的 ON/OFF 事件流，同时原画帧保持完整。相邻两帧同一像素的亮度差越过判决门限时作为事件输出，并定义可视化呈现方式为变亮标红，变暗标蓝。判决门限阈值 θ 与最小变化 ΔL 可在上位机在线调整，也可由板上按键轮换四档预设。

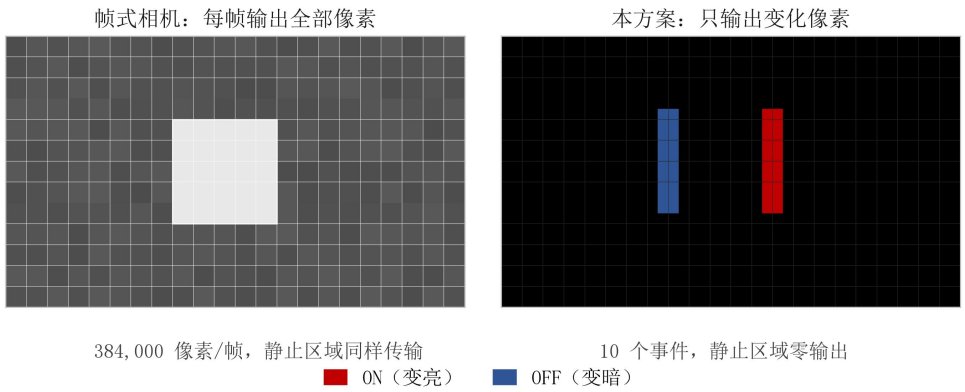


图 1　事件差分示意

事件流分三路输出。一路板上写入事件位图，与原始画面按图层叠加后送板上 LCD 显示。一路千兆网口按 600 包/帧回传，可在「原画+事件」「纯原画」「纯事件」之间切换包内容。一路串口按 1/32 均匀抽点并量化坐标后回传。两路信道都支持全双工与热插拔，串口最高 921600 波特率。

同一条事件流还进入片上 int8 CNN 卷积网络，以每 100 ms 一窗给出「手部开合 / 无动作」的识别结果。结果随下一帧首包回传，可触发上位机录像，亦可在上位机配置为其他行为，同时我们还设计了 API 接口向外暴露识别结果，供外部软件使用。

上位机分四个区域：监控区、调试区、控制区和录制区。支持画面监控、开发调试、板上参数调节、录制导出等功能。

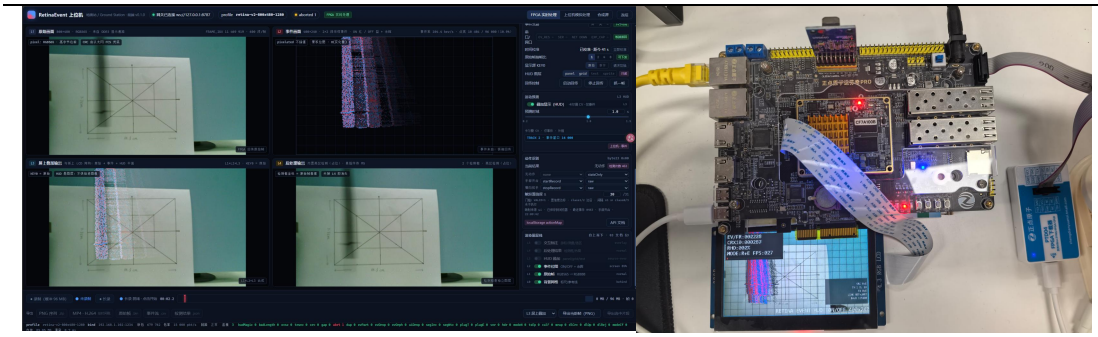


图 2　上位机系统与多屏显示（a）上位机界面（b）板卡与 LCD 显示

1.2 应用领域

本设计适合「大部分时间画面不变、只在少数时刻与少数位置发生变化」的场景。变化越稀疏，收益越大。需要纹理、颜色与文字细节的场合由原画通路补足。

1. 厂区周界与通道监控：静态背景不产生事件，有人或车通过时才回传，链路与存储压力随事件率下降。

2. 设备面板与运行状态监视：指示灯与仪表的变化本身稀疏，一个串口即可长期在线覆盖，不必单独布网。串口版本往下能兼容诸如 STC89C52 等的低性能低功耗单片机，无需网口和 IP 协议栈等。

3. 交通事件检测：车辆经过的时空点直接成为事件，只保存事件记录，不必存整段视频。

4. 人机交互：手部开合在板上二分类即可触发本机录像或其他软件，原始视频不必上传。

5. 设备形态是「事件采集前端」：换一个镜头或换一台主机，同一块板可作为其他嵌入式系统的事件输入。

1.3 主要技术特点

1. 逐像素流式差分，不存整帧：每像素 2 拍完成降采样、对数映射、时间差分与中心-周边判决。事件输出与采集并行，不依赖 DDR3。

2. 一条事件流三处复用：显示位图、网口打包与 100 ms 推理窗共用同一份事件。显示刷新与事件回传互不阻塞。

3. 全片内 int8 网络：50×30×2 输入，三层卷积加 256 维全连接。单窗

207,984 MAC，权重与激活常驻 BRAM，不访问外存。

4. 双信道按带宽裁剪：网口 600 包/帧；串口 1/32 抽点加坐标量化，4 档波特率可在线切换。

5. 显示通路基于厂商 DDR3 与 LCD 例程代码改动二次开发，采集 tap 只读，事件支路不影响原显示链路。

6. 构建与验收脚本化：Vivado 批处理综合实现、ModelSim 仿真、JTAG 下载，每步输出机器可读判据。

1.4 主要性能指标

下表为当前交付位流在 800×480、26.6 fps、室内常态光照下的指标。

表 1　主要性能指标

指标	数值	条件/来源
采集与处理帧率	26.646 fps（37.5 ms/帧）	800×480；实测会话 frames÷span
事件网格与事件率	400×240；2.4~3.1 万事件/秒	室内常态光照；测试位流 v5probe、v7hands
网口回传码率	20.5 MB/s（164 Mbit/s，占 1 GbE 16.4%）	按 600 包/帧×26.6 fps；实测 20.34 MB/s
事件流数据量	≈115 kB/s，约原画 1/171	2.4 万事件/秒×5 B 折算
串口抽样数据量	≈3.4 kB/s，约原画 1/6000	921600 波特、1/32 抽点；网关实测 rxBps
单窗推理	8.80 ms（440,087 拍 @50 MHz）	v5/act3c 基线；v7b 未单独实测
动作识别	动作段召回 48/60 = 0.80；空场景与静止手 360 s 零误触	act8 三轮有效运行
实现资源	LUT 29,266（46.16%）、FF 19,638、BRAM 89.5（66.30%）、DSP 9（3.75%）	act8_uart 交付位流
时序	WNS +0.296 ns、WHS +0.042 ns，0 失败端点	同上

1.5 主要创新点

1. 用常见 CMOS 加数字差分替代专用事件传感器：不依赖 DVS 器件，事件质量由两个可调参数权衡[2-3]。

2. 一条事件流同时驱动显示、回传与片上推理：一次采集三处使用，不需要二次采集。

3. 双信道按带宽裁剪：同一份事件在千兆网口与 921600 串口均可获取。

4. 动作结果两级暴露：板上 TLV 随帧回传，网关再开 HTTP 接口，外部软件可通过暴露的 API 端点取用。

5. 将事件位图、历史亮度和 int8 权重保存在片内 BRAM，由片上 CNN 在 100 ms 决策周期内完成二分类推理。

6. 上位机联动。

1.6 设计流程

实际流程分五个阶段：需求与资源预算、RTL 分模块仿真、单板逐条联通、训练与部署、验收与文档。训练侧在 PyTorch 完成，权重 int8 量化后写入 ROM，上板与离线结果逐窗比对。最后按 A1~A4 流程做板级验收，冻结提交包。

仿真、综合、烧录通过命令行由智能体按技能包自动执行；我们负责开头定目标和验收标准，智能体每轮执行完毕测试板卡、上位机功能和观察相应波形图确认成果或定位问题，反馈智能体进行修复。相关流程示意见图 3。

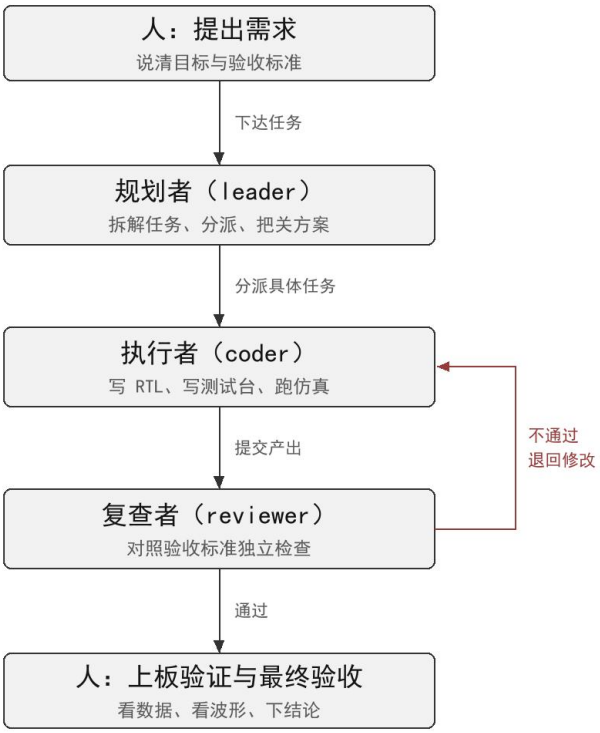


图 3　设计流程

第二部分　系统组成及功能说明

2.1 整体介绍

系统分三层，自左向右是一条单向的数据链。

采集与显示层：OV5640 以 800×480、RGB565、8 位 DVP 输出，像素时钟 48.05 MHz。视频经厂商例程的写通路进 DDR3L，再读出送 4.3 寸 LCD 扫描显示。

事件与回传层：原始像素在引脚处被只读并联出一路（cam_tap），进入事件引擎 retina_core。引擎每两行两列取一个像素，做对数映射后与上一帧同位置相减，得到亮度变化 ΔL；再用中心像素与上下左右四邻的对比度判决是否越过阈值 θ，同时要求 |ΔL| 不小于最小变化，输出事件坐标、极性与时间戳。事件流有三处去向：写进事件位图与 HUD 一起叠加到 LCD；由 eth_udp 打成 UDP 包从底板 GE1 口发出；或由 uart_link 抽点量化后从 CH340 串口发出。

推理与交互层：事件流按 100 ms 窗聚合成 50×30×2 的张量，送 nn_top 做 int8 卷积，得到「手部开合 / 无动作」与 5 位置信度；结果经 act_state_cdc 锁存进 v3 帧头的 byte23，随下一帧首包回传。板上四个按键分别控制显示源、工作点、HUD 图层与回传内容。

模块关系可以概括为：采集是一条只读主干，事件是主干上的旁路，回传与推理都挂在旁路上，显示是主干的末端加事件叠加。四条时钟彼此独立——系统 50 MHz、相机 48 MHz、以太网 125 MHz、LCD 像素 25 MHz——跨域交接全部走异步 FIFO 或格雷码，没有一处依赖相位关系。

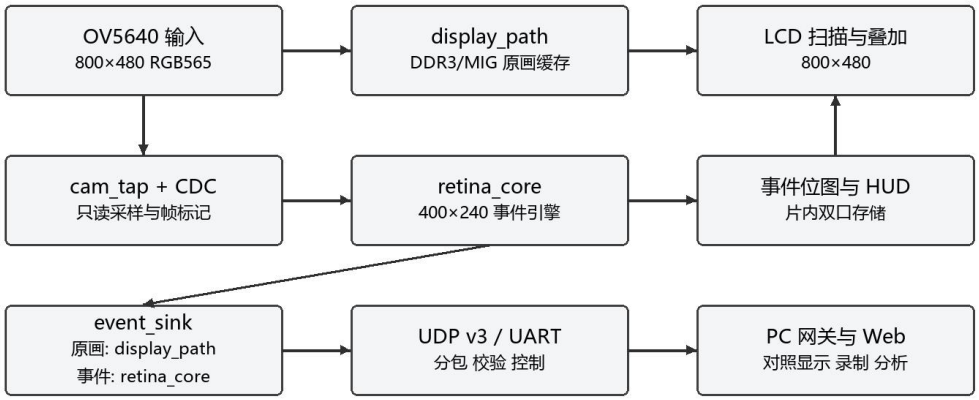


图 4　系统总体功能框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件由厂商核心板与底板组成，主控为 XC7A100T-FGG484-2（Artix-7，63,400 LUT、126,800 FF、135 个 BRAM36、240 个 DSP）。板上提供 DDR3L 显存、4.3 寸 RGB 液晶接口、OV5640 摄像头接口、千兆以太网口、USB 转串口、按键与 LED。原画帧缓存使用板载 DDR3，事件处理状态使用片内存储，引脚与电平配置见 retina_event.xdc[6]。

本作品复用市面上已有的 AMD Artix-7 开发板，硬件工作集中在三处：接口电平与时序（DVP、RGMII、LCD 三组约束）、时钟与复位拓扑、以及供上位机取数的双信道物理连接。资源预算按最终交付位流核算：LUT 29,266（46.16%）、FF 19,638、BRAM 89.5（66.30%）、DSP 9（3.75%），仍有余量加功能。

硬件项	作用
FPGA	XC7A100T-FGG484-2；63,400 LUT、126,800 FF、135 个 BRAM36 等效块、240 DSP。
相机	OV5640，800×480 RGB565，8 位 DVP；两个 PCLK 拼一个像素。
LCD	ATK-MD0430R-800480，800×480，RGB888 并口，像素时钟 25 MHz。
DDR3 / 配置存储	DDR3 承载原画显示缓存；当前交付采用 JTAG 易失配置，Flash 固化验收见 3.6。
外设连接	GE1 千兆网口；JTAG；外接 3.3 V USB-UART；4 个低有效按键与状态输出。

表 2　硬件组成与作用

2.2.2 机械设计介绍

本作品没有需要加工的机械件。整机由开发板、摄像头模组、4.3 寸液晶、网线与 USB 线组成，摄像头用排线固定在板载接口上，液晶插在板载排针座上，长期运行时可用四个铜柱垫高改善散热。

演示时装置水平放置，镜头正对被测区域，距离 0.5~1.5 m。若需固定机位，可用现成云台夹或亚克力支架，不影响电气与约束。

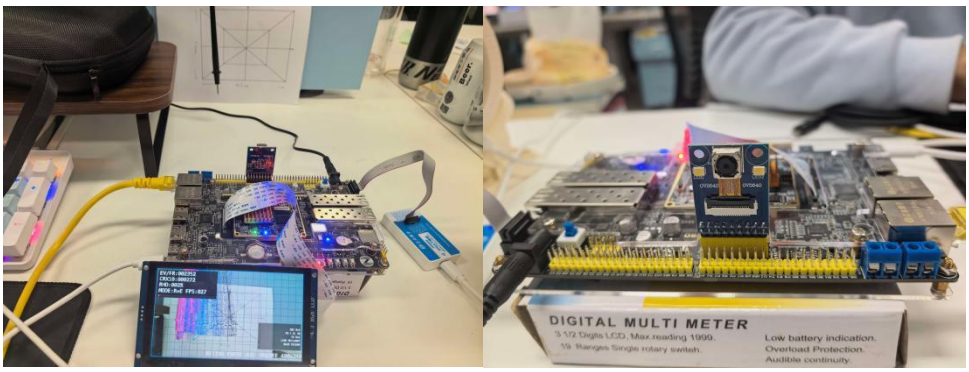


图 5　机械装配与摆放示意

2.2.3 电路各模块介绍

电源与时钟：板卡由 5 V 适配器供电，核心板内部产生各档电压。系统时钟 50 MHz 从 R4 输入，I/O 标准 LVCMOS15；Bank34、Bank35 供 1.35 V，对应 DDR3L。另有相机像素时钟 48.05 MHz、以太网接收时钟 125 MHz、LCD 像素时钟 25 MHz 三路独立时钟，均在 XDC 中单独约束。

摄像头接口：8 位并口 DVP，RGB565，行场同步与像素时钟随线传输。板端只读取数据，不参与配置，SCCB 配置沿用厂商上电流程（800×480 时序）[7]。

显示接口：LCD 模块型号 ATK-MD0430R-800480，4.3 寸、800×480、RGB888 接线；像素时钟 25 MHz，行总长 1056、场总长 525；上电先读屏 ID（0x4384）确认型号再初始化。

存储：DDR3L 作为显示缓存，由 MIG 生成控制器；事件侧不占 DDR3 带宽，一帧事件写完即止。

千兆以太网：使用底板 GE1 口（YT8531C），RGMII 接口，接收时钟 eth_rxc 为 Y18、125 MHz，数据与控制在 LVCMOS33 下约束；核心板另有 GE0

（YT8511SC）口，两个口在本板都实测可用，本工程按 GE1 布约束与接线。

串口：板载 CH340 转 USB，枚举为 COM9，8N1；初始 115200，运行中可切 230400、460800、921600 四档。

人机接口：KEY0~KEY3 接 T4、T3、R6、T6，低有效，20 ms 消抖；LED 指示锁定、显示模式、事件使能与心跳；蜂鸣器是开发板原有器件，保留但不使用。

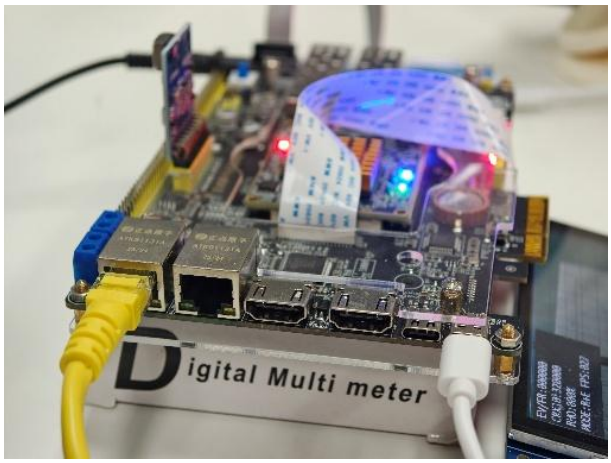


图 6　底板接口区及网线、USB 与供电连接

表 3　关键引脚与电平约束

信号	引脚/参数	电平及意义
sys_clk / sys_rst_n	R4 / U7	LVCMOS15；50 MHz、低有效复位。
key[0..3]	T4 / T3 / R6 / T6	LVCMOS15；20 ms 消抖。
cam_pclk / href / vsync	U18 / N14 / P14	LVCMOS33；数据 8 位独立约束。
uart_rxd / uart_txd	E14 / D17	LVCMOS33；外接 USB-UART。
eth_rxc	Y18，8 ns 约束	LVCMOS33；GE1 RGMII 接收时钟。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分三部分：板上固件、上位机、训练侧。

板上固件全部是 RTL，没有处理器、没有操作系统。综合实现脚本以批处理方式运行 Vivado 2020.2，产物是单一比特流；下载脚本经 JTAG 写入，Flash 固化用 MCS 镜像[4]。

上位机用 TypeScript 写在 Node.js 24 上，分后端网关与浏览器前端。网关负责收包、拆帧、维护状态、对外提供 HTTP/WebSocket 接口；前端负责渲染、调参与录制。两者在同一台机器上分别以进程与页面运行，走本机回环，不额外占网络。

训练侧在 PyTorch 完成：数据集来自板上采集的事件与录制的动作片段，模型量化成 int8 后导出 ROM 初始化文件，随工程一起综合。

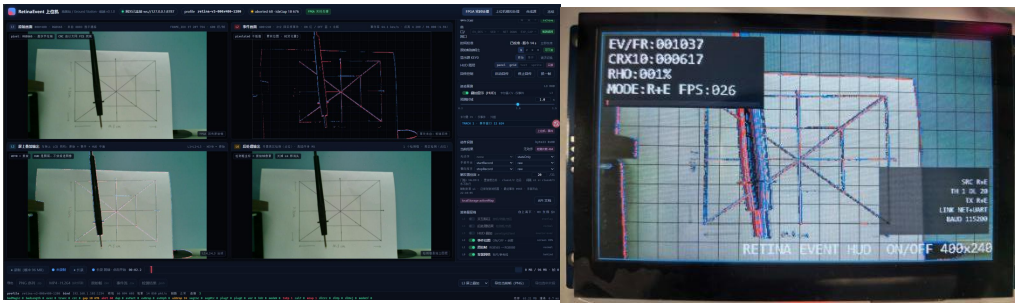


图 7　上位机软件界面与板上 LCD 界面（a）上位机（b）LCD

下图展示了上位机的主要数据流。网关接收并校验报文，将原画分片组帧后分发；浏览器完成图层展示、参数下发和录制导出。原画组帧与事件分发按当前回传模式分别处理。

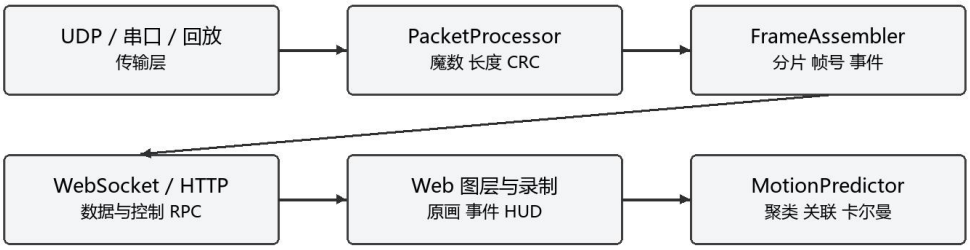


图 8　上位机主要数据流

2.3.2 软件各模块介绍

1. 板端模块与关键接口：

表 4　板端软件模块与关键接口

模块	职责	关键输入	关键输出
retina_core	事件引擎顶层：抽点、对数映射、差分、门限判决	cam_pclk、cam_data、vsync/href	ev_valid、ev_x、ev_y、ev_pol
event_gen	中心-周边判决与不应期	ΔL、θ、minDl	事件脉冲
act_state_cdc	动作状态跨域锁存	nn 结果、50 MHz	byte23（VALID/CONF/CLASS）
nn_top	int8 卷积与全连接	50×30×2 张量	class、conf
eth_udp	打包、校验、ARP、UDP 发送	帧数据、事件、控制回复	RGMII 发送流
uart_link	事件抽点、坐标量化、遥测与 CTRL 帧	事件流、下行字节	串口 TX 字节流
display_path	DDR3 写读、LCD 扫描、HUD 叠加	视频流、事件位图	LCD 像素
hud_stack	网格、事件层、面板、文字、状态五平面合成	各平面像素	合成像素
sys_ctrl	按键消抖与模式切换	KEY0~KEY3	显示源、工作点、图层、回传内容

事件链的流程是：相机像素进 FIFO 跨到 50 MHz 域，按 2×2 抽点保留一个像素，查表完成对数映射，与一帧行缓冲里保存的上次值相减得到 ΔL，再与四邻做中心-周边运算；若超过 θ 且 |ΔL| 超过下限，且距上次事件超过不应期，则输出一个事件。事件写入位图供显示读取，同时进入打包逻辑。



图 9　软件模块划分

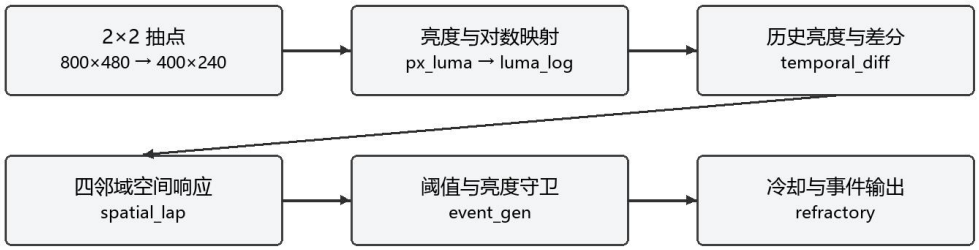


图 10　事件链的六个环节

表 5　四档事件过滤预设参数

档位	θ	MIN_DL	用途
0	1	20	上电默认，运动区域
1	1	32	区域省流
2	3	18	运动边缘
3	44	1	历史对照

2. 上位机模块与关键接口：

表 6　上位机软件模块与关键接口

模块	职责	关键输入	关键输出
transport/udp	收 UDP 分片并校验	端口 1234	帧块
transport/serial	打开 COM 口、按 v3 帧切分	波特率、COM9	事件帧、遥测帧
pipeline/processor	还原帧、解码事件、更新动作快照	帧块、事件块	Frame、EvBatch、Action
action	维护动作快照与事件序列	v3 头 byte23、TLV 0x92	/api/action
ipc/ws	对外 WebSocket 与 HTTP	内部状态	前端消息
web/ui	面板、图层、参数、录制	WS 消息	交互事件
predict/motion	卡尔曼预测叠加	事件窗口	预测框
recorder	录像与 CSV 导出	帧、事件	mp4、csv

上位机数据流的流程是：网关收到 UDP 包后按帧号拼接，校验通过即交给处理器；处理器把原始帧解码成 RGBA 供前端显示，把事件块解码成坐标数组供事件层渲染与预测；动作快照由 v3 帧头的 byte23 或 GET_STATUS 应答里的 TLV 0x92 更新，前端每 500 ms 轮询一次快照，事件则走长轮询增量拉取。串口链路复用同一套处理器，差别在帧切分与降采样参数。

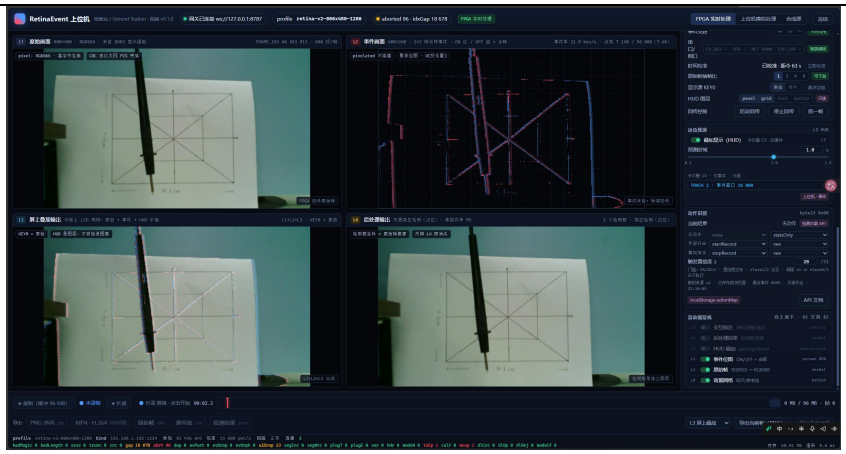


图 11　上位机主界面分区

表 7　上位机界面分区与功能

区域	功能	说明
一区	连接状态	板卡与电脑的连接情况
二区	通信信道选择	网口回传原画、事件和板端参数；串口对事件均匀抽点（默认每 32 条取 1）并量化坐标
三区	统计	一秒内的事件数，以及事件相对原画的数据量降低倍数
四区	参数控制	实时调整 θ、最小变化量、曝光上限，控制回传内容和是否处理事件
五区	运动预测	基于卡尔曼滤波的事件轨迹与未来位置预测
六区	动作检测	显示板上 CNN 的识别结果，可作为录制或触发的信号
七区	HUD 图层	控制监控画面上 HUD 各图层的显示

此外我们还在 Web 端加入事件聚类和卡尔曼匀速预测，用 HUD 叠加轨迹与未来。图 12 使用合成事件源展示该功能，合成源和单元测试的验证范围

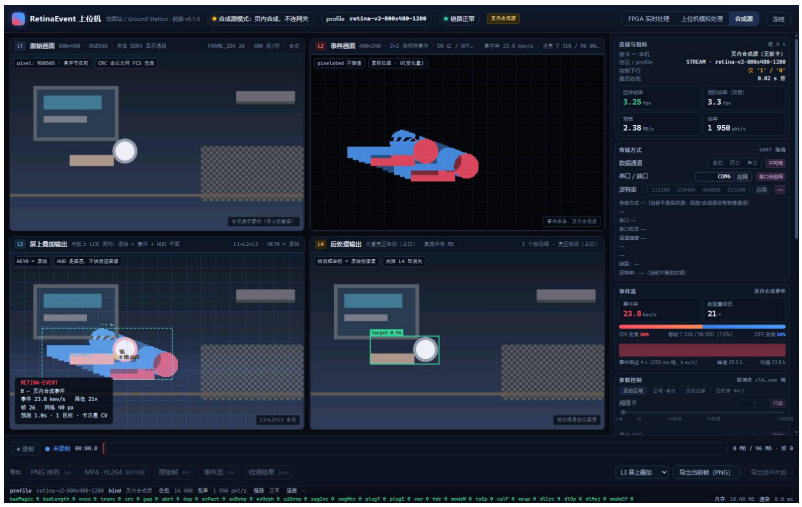


图 12　合成事件源的运动预测演示

3. 接口与通信协议：UDP 1234 收板端数据；HTTP/WebSocket 8787 供前端

与控制；REST 提供 /api/action、/api/action/events、/api/frame.json、/api/stats、/api/health；串口 8N1，115200~921600。

表 8　接口契约

接口	信号或记录	契约
VPS 像素	data[15:0]、de、vsync	de 可稀疏；只在有效像素时推进；vsync 重置帧状态。
EVS 事件	valid、x[8:0]、y[7:0]、pol	同拍有效；跨域时整个字走 FIFO。
位图读口	addr[15:0]、data[7:0]	LCD 域同步读；一个地址含两个像素。
OVL 叠加	有效/遮罩与 RGB	hud_stack 按优先级合成，hud_inject 接到扫描输出。
参数/命令	模式、阈值、更新 toggle	参数字用 cdc_tog_word 跨域；写入后以实际回读确认。

UDP v3 上行报文采用魔数 F0 5A A5 30、版本 03 和 24 B 包头。原画每块 1,280 B，每帧 600 块，RGB565 按高字节在前传输。包头中的多字节整数使用大端编码；5 B 事件记录则按自定义格式排列：b0 为 X 低 8 位，b1 为 X 高 1 位，b2 为 Y，b3 为 POL，b4 为 FLAGS。解析事件 X 坐标时须按该格式重组。[4]

表 9　UDP v3 包头字段

偏移	字段	长度与含义
0～7	MAGIC / VER / HDR_LEN / FLAGS / MODE	4+1+1+1+1 B；模式 0 原画+事件、1 原画、2 事件。
8～15	CHUNK_IDX / FRAME_IDX / RAW_LEN / EV_LEN	各 2 B；分片、帧号、两段长度。
16～23	TS_US / EV_REC_BYTES / ACT	6+1+1 B；微秒时间戳、记录宽度、动作字节。
24 起	RAW_DATA / EV_DATA	按头中长度顺序排列；CTRL 包有独立控制载荷。
尾部	CRC16	2 B；CCITT-FALSE，poly=0x1021，init=0xFFFF。

包头 offset 23 用于动作回传：bit7 为 VALID，bit6～2 为 CONF，bit1～0 为 CLASS。CONF 为 min(margin,31) 的整数评分，不是概率。该

字节保存最近一次动作判定，连续 1.0 s 无新脉冲后归零；打包器每帧锁存一次，同一 FRAME_IDX 的各数据分片保持一致，并参与 CRC 校验。PC 侧通过 GET /api/action 或 TLV 0x92 回读状态。

在 1,472 B UDP 负载上限内，混合包最多包含 33 条事件，纯事件包最多包含 289 条，最大数据包为 1,471 B。包括 CTRL 包在内，打包器均等待发送器 ready 后启动。EV_EN 只控制事件支路，关闭事件时原画仍可上行。我们分别统计缺片、CRC 错误、事件丢弃和 UI 丢帧，避免混用不同环节的计数。时间戳由打包域生成或校准，不代表像素级独立感光触发时刻。

以太网配置为板端 192.168.1.10、PC 端 192.168.1.102，上行 UDP 端口为 1234，下行命令端口为 1235。命令包括 SET_MODE、SET_PARAM、TIME_SYNC、GET_STATUS 和 EV_EN，ACK 通过 CTRL/TLV 返回。UART-LINK 复用相同控制语义，在帧边界按 CTRL、遥测、事件帧的优先级仲裁，支持 115200、230400、460800、921600 四档波特率。我们通过坐标量化、抽点和聚合适配串口带宽，由串口承载事件与遥测，由千兆网口承载原画等较大数据流。

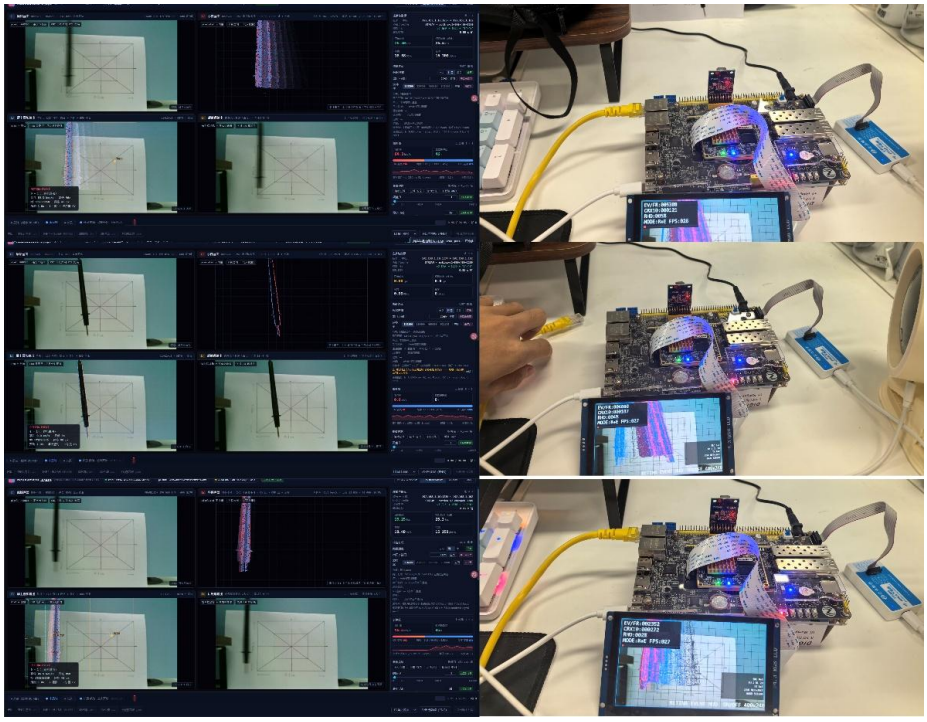


图 13　板端热插拔功能实现与上位机联调实拍

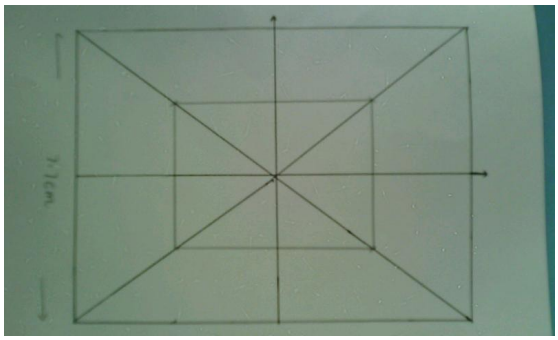


图 14　UDP 相机直拍回传画面

4. 时钟域与存储组织

表 10　时钟域划分与跨域方式

时钟域	频率	主要对象与跨域
cam_pclk	48.019 MHz（ILA 实测；约束 20.833 ns）	DVP 组包；17 位 FIFO 到事件域和网络域。
clk_cam	50 MHz	事件链、按键和位图 A 口；事件 18 位 FIFO 到网络。
clk_lcd	25 MHz	LCD、位图 B 口、HUD；统计逐位同步仅用于显示。
eth_rxc / clk_eth	125 MHz	UDP 组包、时间戳、命令；控制参数用字同步跨域。
MMCM 参考	200 MHz	MIG 等参考；按 IP 的实际配置与实现报告核对。

我们按容量和访问带宽分配存储。800×480 RGB565 原画单帧为 768,000 B，即 6,144,000 bit，使用 DDR3 缓存；历史亮度为 768,000 bit，事件位图为 384,000 bit，冷却位图为 96,000 bit，另有行缓存、网络权重和 CDC FIFO，均使用片内存储。实际 BRAM 占用以 Vivado 利用率报告的 Block RAM Tile 为准，不能仅按逻辑位数折算。

第三部分　完成情况及性能参数

3.1 整体介绍

整机形态是一块开发板、一路摄像头、一块 4.3 寸液晶、一根网线与一根 USB 线。上电后数秒内出现 LCD 画面，左上角 HUD 显示事件数、压缩倍数与活跃率，右下角显示来源、阈值、链路与波特率。接上网线后，上位机打开即出画面；插上 USB 后，串口链路可单独回传事件与遥测。位流已经固化在板上 Flash 中。

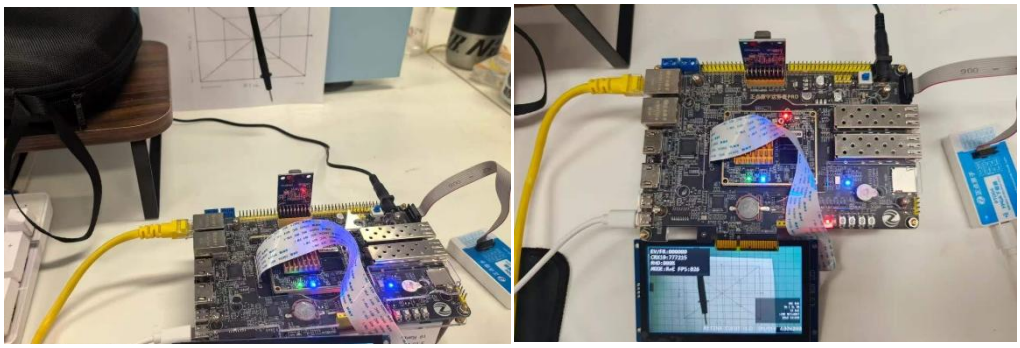


图 15　整机实物（达芬奇 Pro + OV5640 + 4.3 英寸 LCD + 千兆网口 + 串口）

3.2 工程成果

3.2.1 机械成果

本作品无自加工机械件，实物即板卡与现成模组的组装形态，装配方式见 2.2.2。

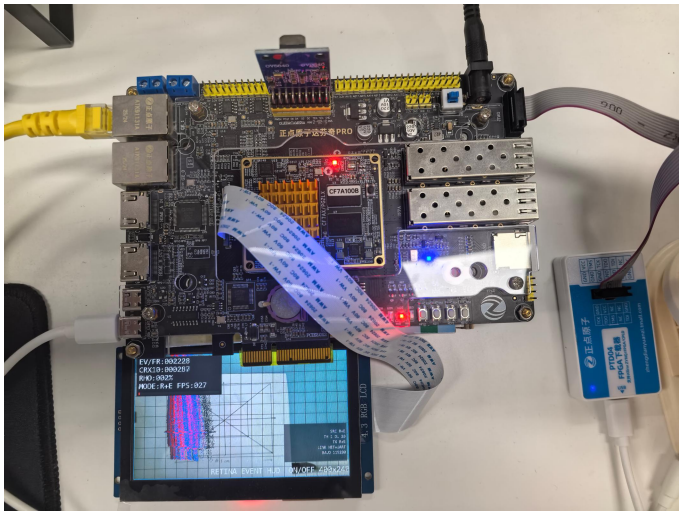


图 16　成果照片

3.2.2 电路成果

电路成果为厂商核心板与底板的组合，以及本作品所需的四组接口连接：摄像头排线、LCD 排线、网线（底板 GE1）与 USB 串口线。供电为 5 V 适配器。

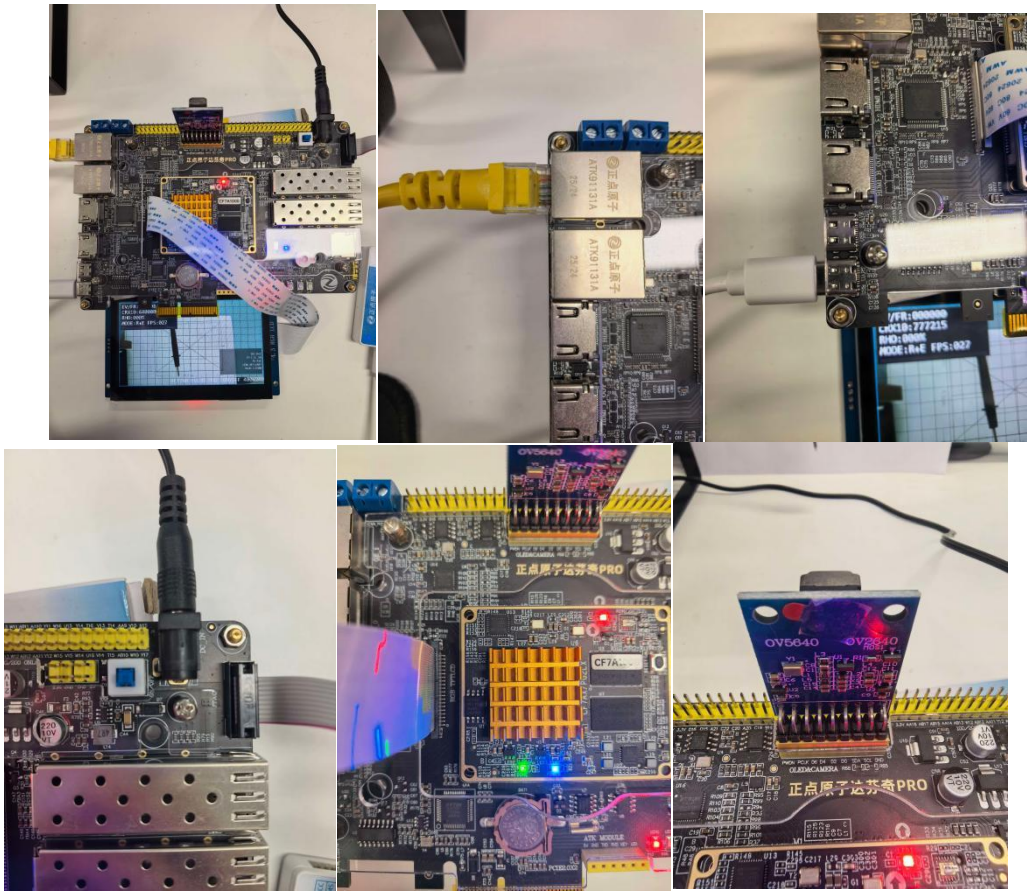


图 17　板卡与接线实拍（a）整体（bcdef）接口局部

3.2.3 软件成果

1. 板上成果

板上固件给出单一交付位流与配套报告：综合实现日志、时序报告、资源报告、DRC 报告、位流 md5。构建与验收全部脚本化，可在另一台机器上从源码复现。

我们以 ETH_EN=1、v7b 权重构建 act8，并采用 Explore 布局策略；表 11 的资源及时序来自 variant_act8_final 的 util.rpt 和 timing.rpt。历史 G 版为启用 UART_LINK_EN=1 的事件引擎配置，占用 26,334 LUT、80.5

BRAM，不能与 act8 直接混用。表 12 另列 act3c、act7、act8 的动作识别版本演进。

表 11　act8 实现资源占用

资源	act8 使用	器件总量	占比
Slice LUTs	22,383	63,400	35.30%
FF	14,325	126,800	11.30%
Slice	8,026	15,850	50.64%
BRAM36 等效块	86.5	135	64.07%
DSP48E1	8	240	3.33%
IOB	140	285	49.12%

利用率报告中的 BRAM 86.5 为 Block RAM Tile，即 RAMB36 + RAMB18/2。act8 的 WNS=+0.215 ns、TNS=0，建立检查失败端点为 0/84,390；WHS=+0.017 ns、THS=0，保持检查失败端点为 0/84,367。Slice 占用为 8,026/15,850，即 50.64%，尚余 7,824 个。我们将在后续扩展中同时检查资源增量、布局拥塞与时序裕量。

图 18 和图 19 是我们在 Vivado 2020.2 中导出的全功能工程视图。图 18 按层次列出 LUT、寄存器、BRAM 和 DSP 占用；图 19 展示实现后的器件布局及高亮区域。该截图工程同时启用 UDP、UART-LINK 和动作识别，顶层占用为 29,266 Slice LUTs、19,638 FF、9,990 Slice、89.5 Block RAM Tile 和 9 DSP。uart_ev_tap 层合计占用 5,810 LUTs、3,793 FF，其中 uart_link 本体为 765 LUTs。该分支是其相对 act8 资源增加的重要来源。

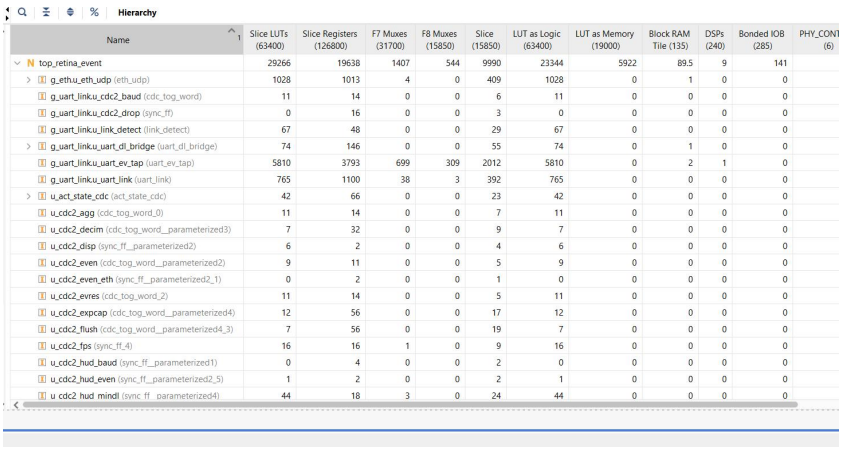


图 18　层次化资源占用（Utilization 视图）

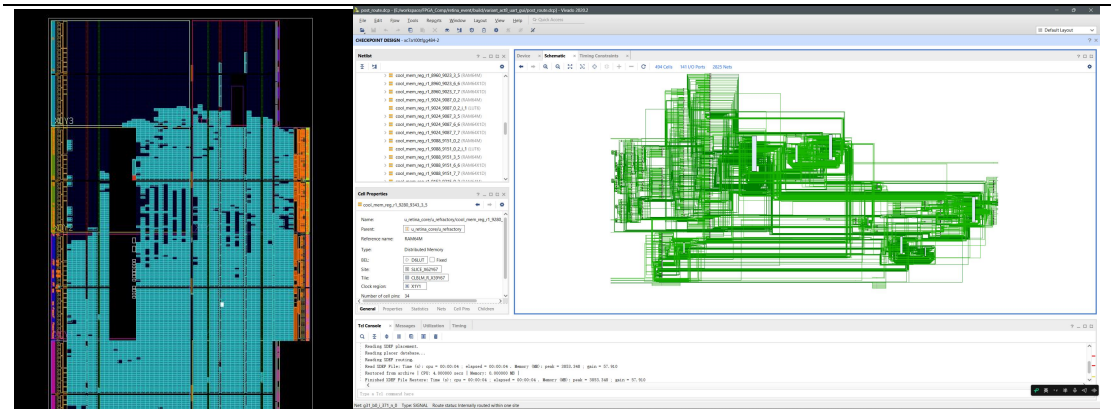


图 19　实现后的器件布局（高亮为所选对象的器件分布）

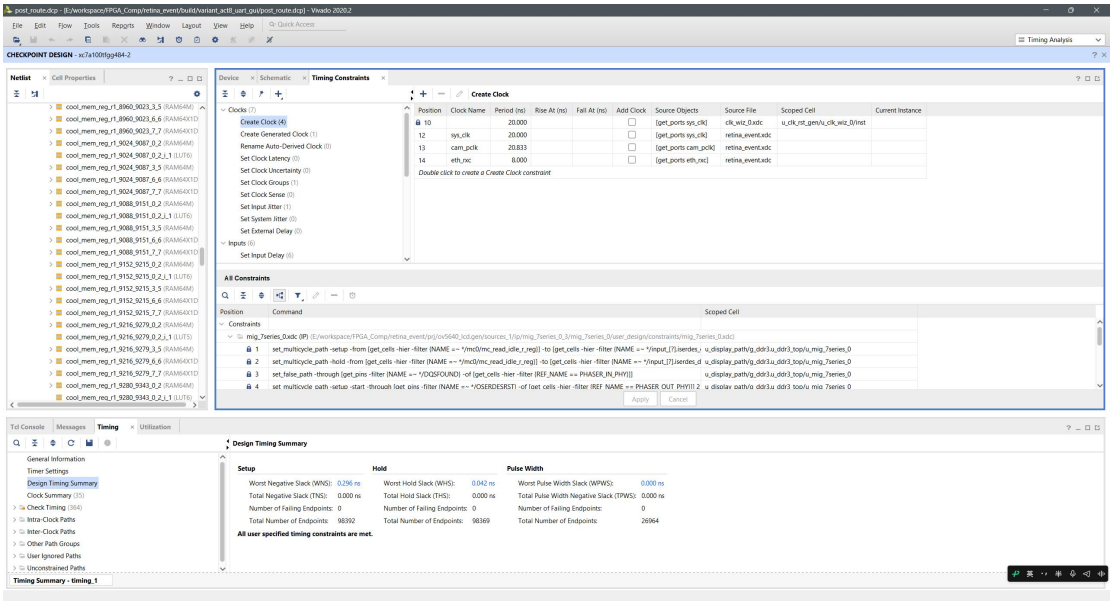


图 20　实现后时序分析报告

上述两图用于说明全功能工程的层次资源与布局情况；交付 act8 的资源仍以表 11 为准。器件视图的高亮表示当前所选对象在器件上的分布，资源余量应读取利用率报告。图 21 对比 act3c、act7、act8 三代实现的六项资源占用，表 12 给出对应绝对量及板级状态。

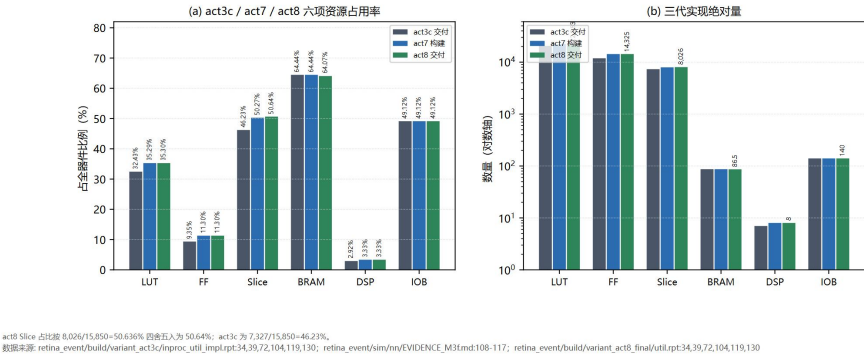


图 21　三代实现资源占用（act3c 交付 / act7 构建 / act8 交付）

表 12　act3c / act7 / act8 三代实现对照

版本	LUT	FF	Slice	BRAM	DSP	WNS / WHS	板级状态
act3c	20,559	11,850	7,327（46.23%）	87	7	+0.306 / +0.036	已烧板（动作实测）
act7	22,371	14,325	7,968（50.27%）	87	8	+0.059 / +0.027	已构建，未烧板
act8	22,383	14,325	8,026（50.64%）	86.5	8	+0.215 / +0.017	当前板载

从 act7 到 act8，我们替换为 v7b ROM、将 margin 门限设为 16，并采用 Explore/phys_opt 实现方案；LUT 增加 12，FF 不变，Slice 增加 58，BRAM 减少 0.5，WNS 由 +0.059 ns 提升至 +0.215 ns。与 act3c 相比，act8 的动作前端和决策链继续演进，BRAM 从 87 变为 86.5，DSP 增加 1 个，Slice 占比由 46.23% 升至 50.64%。图 22 展示各阶段的时序收敛结果。

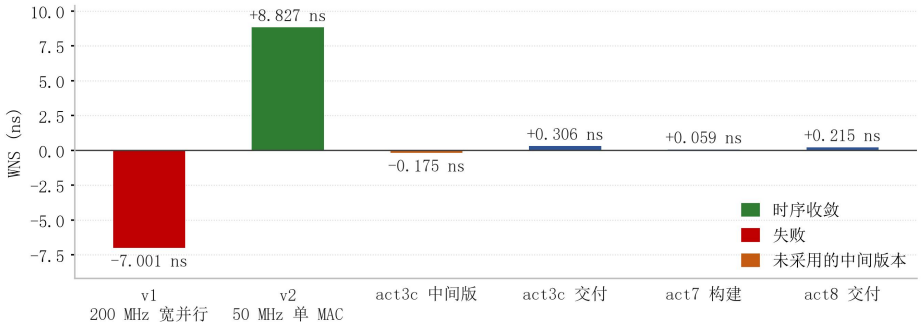


图 22　各阶段实现的最差建立裕量（WNS）

我们将三次关键缺陷及修复结果汇总于表 13：H63 涉及抽点接口约定，MTU 涉及打包长度上限，F→G 涉及帧标记跨域与错相恢复。这些问题说明，常规功能仿真之外，还需要真实相机时序、异常激励和原始报文检查。

表 13　关键缺陷与修复对照

优化案例	修改前	修改后及证据
采样契约 H63	每帧进入空间算子的有效数 383,999；抽点失效。	恢复 96,000；新增相机通路测试及 2,390 条事件对拍。
UDP 排空路径 MTU	混合包可超 1,472B，出现截断/CRC 丢失。	覆盖排空路径和硬钳位；对应失效 TB 转 PASS。
F→G 帧标记跨域	丢像素后仅靠本地计数可永久错相。	帧首 marker 随像素过 FIFO；LUT+35、FF+3，BRAM/DSP 不增；WNS +0.332→+0.243ns。

G 版烧录后的两次快照显示包率为 15,963/16,085 包/s，网关吞吐为 20,523,615/20,743,924 B/s，badMagic、badLength、crcFail 等计数均为 0。板端 dropEv 累计值为 7,178，两次读数之间未增加；网关 evDrop 为 0。这表明观测窗口内未发现对应计数所覆盖的新增丢弃，不能据此排除所有链路损失。吞吐按 UDP 应用负载统计。[7]

每帧 N 条事件的净荷为 5N B，原画净荷为 768,000 B，因此二者的数据量比为 768,000/(5N)。该比值不含包头、CRC 和链路开销；切换 EV_ONLY、停止原画回传后，才能体现纯事件传输的带宽收益。事件数还受场景变化、噪声、阈值及传输抽点影响，应在同场景下统计比较。时延采用第 3.5.3 节的分类结果。

2. 2. 上位机成果

上位机给出可运行的多端界面：左侧原始画面与事件层对照，右侧连接状态、信道选择、事件率与带宽倍数、参数与录制；另提供动作卡片、卡尔曼预测开关与 HUD 图层控制。

我们在上位机中实现了原画与事件多窗格展示、协议统计、板参回读与下发、事件参考计算、录制和按数据类型导出。图 23 是板载 UDP 模式的联调截图，四个窗格依次展示原画、去噪前事件、HUD 叠加及去噪后结果。右侧可见板端地址 192.168.1.10:1234、STREAM / retina-v2-800x480-1280 配置、回传帧率 25.75 fps、相机帧率反推值 25.8 fps、带宽 18.86 MB/s 和包率 15,450 包/s。该时段事件率均值为 33.7 k 事件/s，近 4 s 峰值为 52.0 k；当帧事件数为 1,310，占 96,000 网格点的 1.4%。界面显示的 117× 为该帧原画与事件净荷之比，不表示包含原画回传时的实际网络带宽降低。截图读数与表 14 的长录数据分别对应不同观测条件。

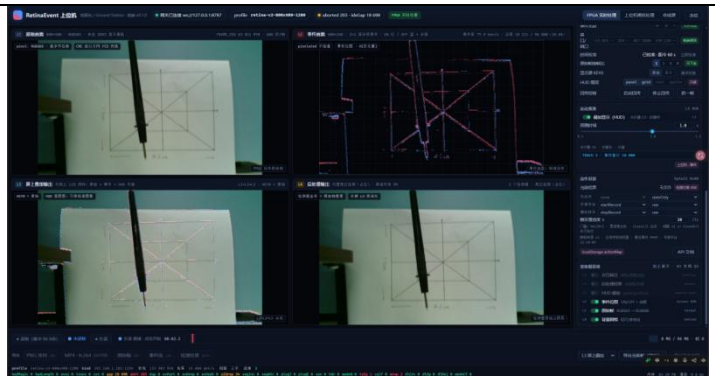


图 23　上位机整体界面

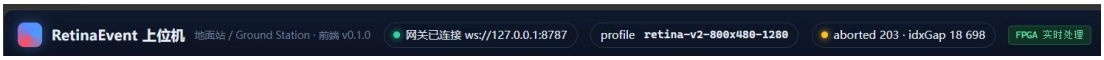


图 24　网关连接状态与模式选择



图 25　事件处理与页面切换控制

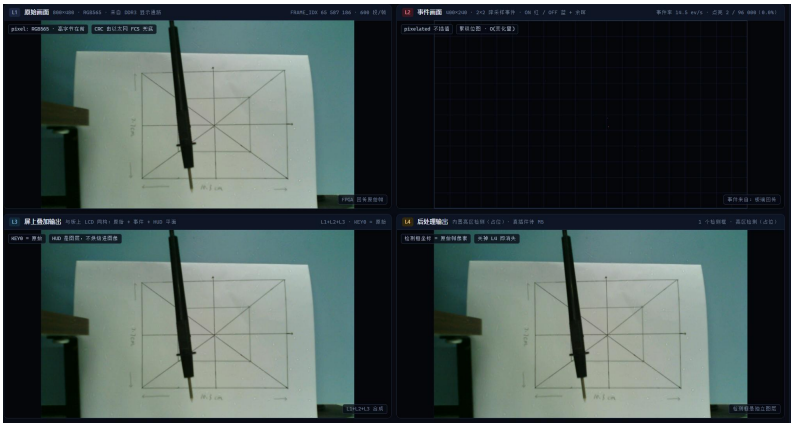


图 26　监控区

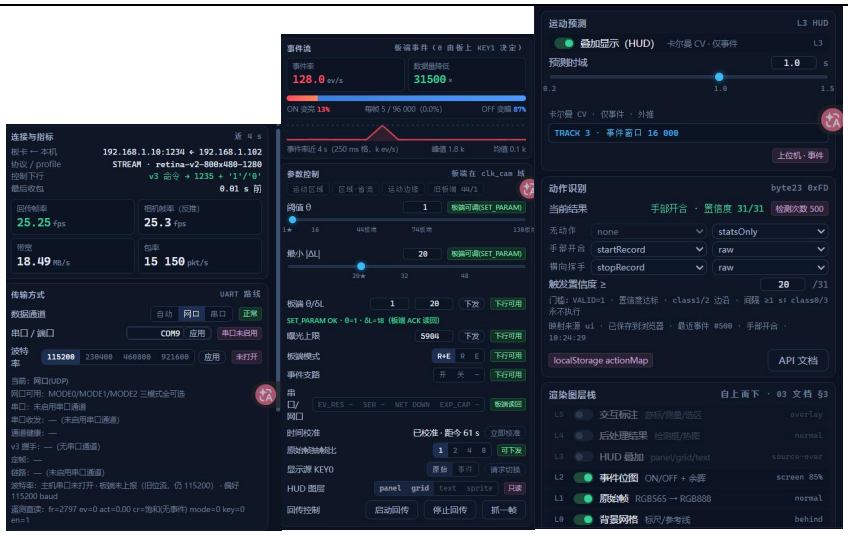


图 27　参数控制区



图 28　导出控制台



图 29　实时数据监看

在开发中，我们将上位机的开发和板卡解耦开发，因此专门设计了一个合成源页面，供调试和新功能开发验证使用。

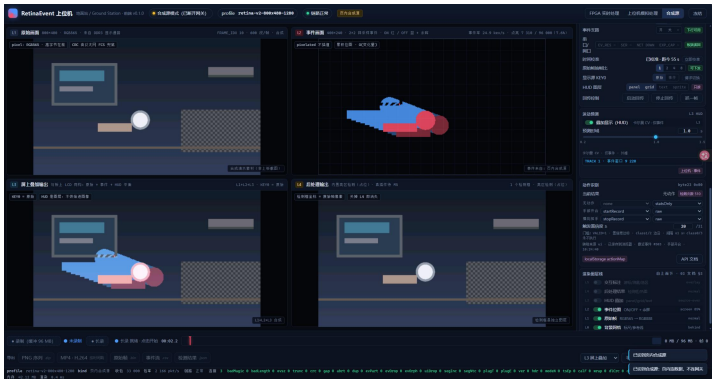


图 30　合成源调试页面

3.3 特性成果

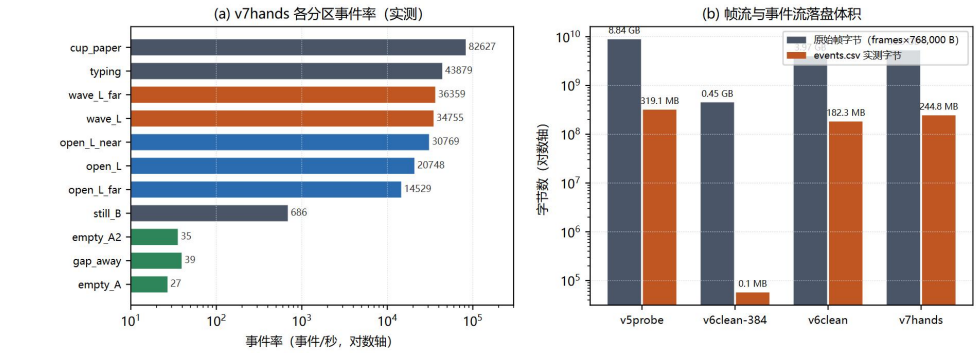
表 14　特性成果量化

特性	量化结果	证据
帧流转事件流	800×480 → 400×240，26.646 fps	实测会话 frames÷span
事件率	2.4~3.1 万事件/秒（室内常态）	v5probe、v7hands 采集记录
网口回传	20.5 MB/s，占 1 GbE 16.4%	600 包/帧 × 26.6 fps；实测 20.34 MB/s
事件流压缩	≈115 kB/s，约 1/171	2.4 万事件/秒 × 5 B
串口回传	≈3.4 kB/s，约 1/6000	921600 波特、1/32 抽点，网关实测
显示	800×480、25 MHz 像素时钟，HUD 五平面叠加	LCD 扫描实测
片上动作识别	动作段召回 48/60 = 0.80；空场景与静止手 360 s 零误触	act8 三轮有效运行
单窗推理	8.80 ms	v5/act3c 基线
资源	LUT 46.16%、BRAM 66.30%、DSP 3.75%	act8_uart 位流
时序	WNS +0.296 ns、WHS +0.042 ns	同上
复现	一条命令构建，判据机读	构建脚本与日志

3.4 测试与验证方法

3.4.1 事件流与码率

我们使用三段历史长录统计事件流：v5probe 为 432.143 s、11,516 帧、10,312,327 条事件；v6clean 为 194.356 s、5,174 帧、6,012,765 条事件；v7hands 为 256.088 s、6,831 帧、7,913,893 条事件，录制统计中的丢帧、丢事件均为 0。这些数据采于 act8 烧录之前，用于分析事件链负载。图 31 左侧是选定场景段的平均率：空场景约 27～39 事件/s，开合约 1.5～3.1 万，挥手约 3.5 万，打字约 4.4 万，纸杯/纸张约 8.3 万；未计入图中的相机转动段最高约 13.4 万，因此 82,627 不是全程峰值。右侧区分原画体积与事件 CSV 体积：v5probe 分别为 8.84 GB 和 319.1 MB，同段事件按 5 B/条折算的二进制净荷约为 51.6 MB，CSV 体积约为其 6.2 倍。

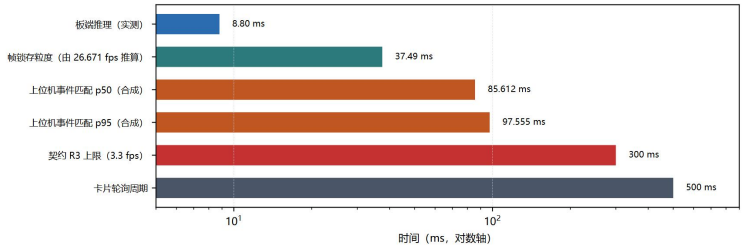


数据来源: records/*/meta.json + frames.csv + events.csv；report/figures/v7_zones.json；统计脚本 report/figures/stream_stats.py（快照 report/evidence/event_stream_stats.json）

图 31　所选场景段平均事件率与落盘体积

3.4.2　端到端时延

我们分别分析推理、帧锁存、上位机匹配与轮询耗时，图 32 将这些时间放在同一坐标轴上，并不将它们相加作为端到端结果。其中，推理引擎测试台计数折算约为 8.80 ms；37.49 ms 由 26.671 fps 推算，仅表示帧周期与帧锁存粒度。历史上位机匹配记录给出 p50 85.612 ms、p95 97.555 ms，但仅含 4 个匹配样本。动作卡片的 R3 契约上限为 300 ms，轮询周期为 500 ms，二者也属于不同定义。图中推理项的“实测”指测试台周期计数，并非 act8 整机计时；当前尚无本版完整端到端时延实测。



数据来源: EVIDENCE_M3.md:29；records v7 fps；EVIDENCE_EVAL.log:51；契约 §8 R3:462

图 32　端到端时延节点（实测 / 推算 / 契约，非同一链路求和）

3.4.3　动作识别评测

我们以 2026-10-30 采集的 256.088 s 事件数据构建动作数据集，共含 6,831 帧、7,913,893 条事件，录制未报告丢帧。训练侧使用 100 ms 窗、50 ms 步长，剔除间隔段和两段转动区间后保留 4,589 窗，并对相应片段做左右镜像增强。含 7 min 对照段的标注文件共 14,447 行。图 33 分别展示可用窗口与标注行数，两种数量不混用。

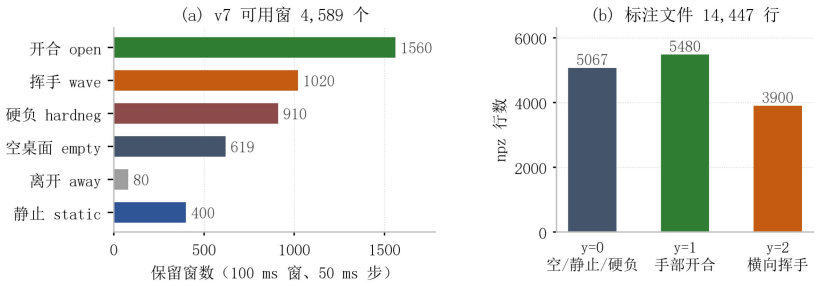
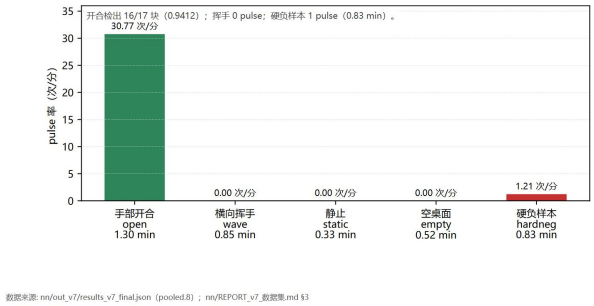


图 33　动作数据集构成：分组窗数与标注行数分布

图 34 展示 v7 基线 float 模型在 margin=8、N_CONSEC=3 下的留出结果：开合为 30.77 次/分（1.30 min），横向挥手、静止和空桌面均为 0，硬负样本为 1.21 次/分（0.83 min）。该结果属于训练侧 s50 评测。部署模型 v7b 另按 s100 非重叠窗扫描门限：8/3、16/3、24/4 三组中，16/3 保持开合 17/17，将硬负 pulse 从 3 降至 2、总 pulse 从 88 降至 74，因此被选为部署参数；24/4 虽将硬负降至 1，但开合降为 14/17，未采用。



数据来源: nn/out_v7/results_v7_final.json（pooled.8）；nn/REPORT_v7_数据集.md §3

图 34　留出集各动作段 pulse 率（float 权重，margin=8，N_CONSEC=3）

我们连续执行了 4 轮 A1～A4 分段测试。第 1 轮 A4 段双手未离开画面，操作无效；其余 3 轮用于统计，A1、A2 各累计约 180.4 s，均未误触发，A3 分别检出 14/20、19/20、15/20，累计召回 48/60=0.80，A4 出现 1 条退场过渡事件。需说明，本轮采用约 182 s 的简化流程，阶段时长为 60/60/40/20 s；正式方案为 60/60/70/20 s，共 210 s，本轮未执行 A3 动作后的 30 s 静止保持段。因此“3 轮操作有效”不等于正式方案验收有效，4 轮 VERIFY_PASS 均为 0。我们保留这些分段结果，后续按完整流程复验；当前召回尚未稳定达到 G2（≥0.85）。

表 15　act8 上板 A1～A4 简化分段测试结果

轮次	A1 误触	A2 误触	A3 召回	A4 误触	判定
1（操作无效）	0	0	9/20 = 0.450	6	A4 手未出画，协议未执行

轮次	A1 误触	A2 误触	A3 召回	A4 误触	判定
2	0	0	14/20 = 0.700	0	召回未达 0.85
3	0	0	19/20 = 0.950	1	A4 退场过渡 1 条
4	0	0	15/20 = 0.750	0	召回未达 0.85

我们将以下情况纳入回归检查：首帧抑制、稀疏与连续 de、输入源切换、空间边界及有符号右移、FIFO 空满、CRC 与 MTU、帧号回绕及丢片、三种回传模式切换、串口帧边界及切速、纯事件录制和不同原画选区的导出，以及动作窗边界、连续确认与不应期。后续回归沿用这些检查项；针对关键修复保留旧实现作负对照，确认测试能够识别原有缺陷

3.5 联机、操控与回传

3.5.1 热插拔与全双工

两条信道都能在运行中插拔，不需要重启上位机，也不需要重新烧写。网关每 3 s 判定一次通道健康：当前通道在 3 s 内没有新的字节或数据包，状态标为 degraded 并重新探测。网口重探测时先释放 UDP 端口再重新绑定，避免“绑定成功但收不到包”的假在线；恢复收包后自动回到 active。串口打开后发送一帧 GET_STATUS，300 ms 内收到 ACK 即确认链路与位流版本，失败则试满 3 次再上报。

两路都是全双工。网口上行由 1234 端口承载原画、事件与遥测，下行由 1235 端口承载 SET_MODE、SET_PARAM、TIME_SYNC、GET_STATUS、EV_EN 等命令，ACK 经 CTRL/TLV 返回；串口用同一对 TX/RX 承载同样的命令语义，板端在帧边界按 CTRL、遥测、事件帧的优先级仲裁。插拔过程中已建立的下行命令不会串到新连接的帧里。

承载能力按信道分档。千兆网口实测 20.5 MB/s，即 164 Mbit/s，约占 1 GbE 的 16.4%，一帧 800×480 RGB565 原画连同事件一次发完；串口四档 115200、230400、460800、921600，按 1/32 均匀抽点并量化坐标，921600 下事件码率约 3.4 kB/s，可直接挂到只有串口的低端设备上。两种信道同时在线时，上位机按显式选择使用其中一路，另一路保留但不混用数据。

3.5.2 板上操控按键

板上四个机械按键的低有效信号经 20 ms 消抖后才产生一次按下脉冲。sys_ctrl 在 50 MHz 域把每次脉冲翻译成一次状态更新：显示源、工作点档位、HUD 图层预设与回传内容各占一个寄存器，更新后同时送达显示域与打包域。状态机是“按下—锁存—生效”三段：按下沿到来后先把新的选择写入寄存器，再由同一个时钟把它推给下游，按住不放不会反复翻转。

为让上位机看到同一份状态，网关每 500 ms 轮询一次 GET_STATUS，把 TH、DL、SRC、TX、LINK、BAUD 与动作字节一起读回。因此按键后最多半个轮询周期，面板参数与右下角 HUD 就同步显示。本文 Key1~Key4 对应板上丝印 KEY0~KEY3。

（1） 显示源切换（丝印 KEY0）

按下后显示源在原画与事件画面之间切换，LCD 右下角 HUD 的 SRC 字段与上位机监控区标题同步变化；再按一次切回。

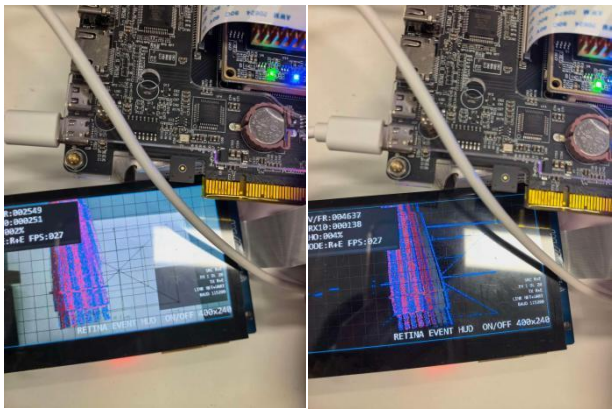


图 35 初始状态与按下 Key1 前后的显示源切换

（2） 工作点轮换（丝印 KEY1）

每次按下工作点轮换一档，θ 与最小 ΔL 依次为（1,20）、（1,32）、（3,18）、（44,1）。前两档保留连通运动区域，第三档只留碎片化边缘，第四档接近全黑；事件率随之从约 3 万事件/秒降到接近 0，上位机参数区与 HUD 的 TH、DL 当帧刷新。

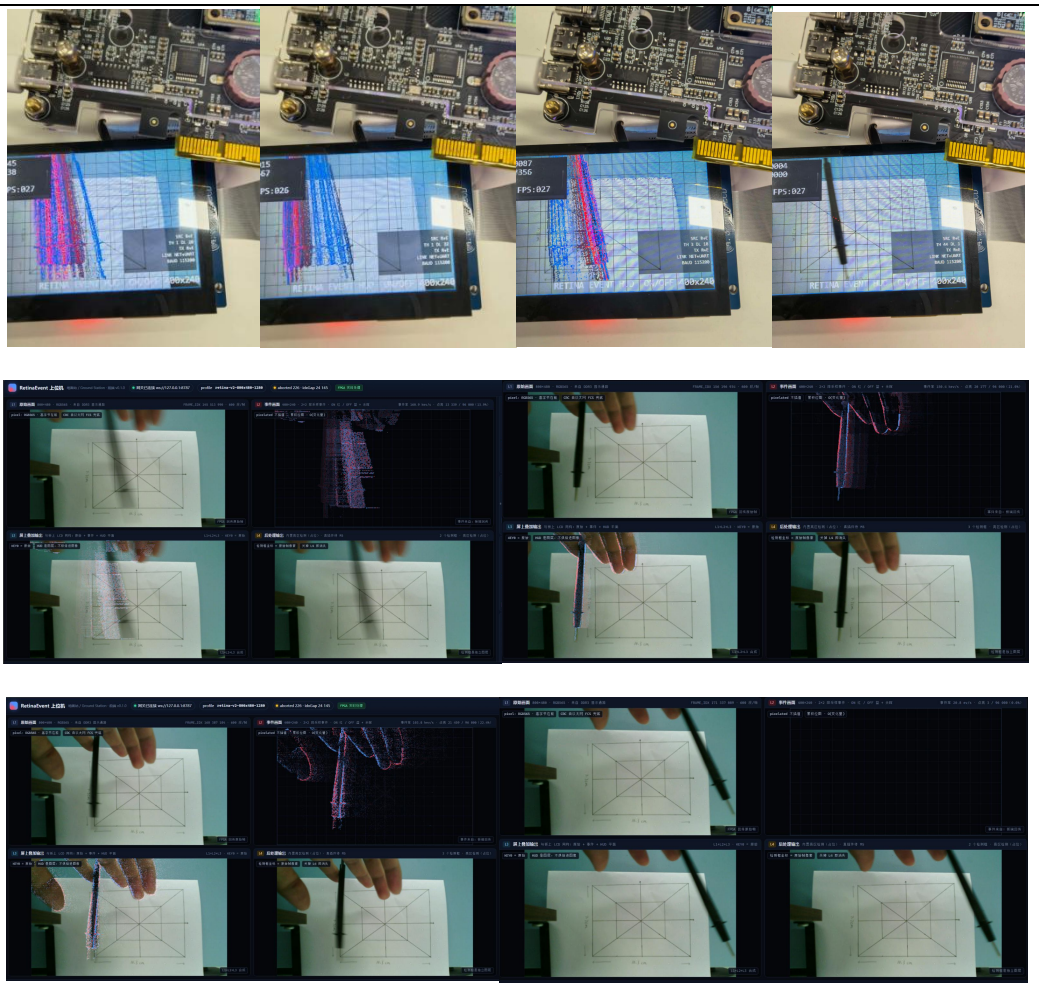


图 36　工作点四档轮换：板上 LCD 与上位机参数同步变化

（3）　HUD 图层逐级关闭（丝印 KEY2）

每次按下关闭一层 HUD，顺序是事件层、面板、文字，最后只剩网格；再按一次恢复四层全开。上位机监控区的叠加显示同步减少，便于判断是哪个平面在影响读数。

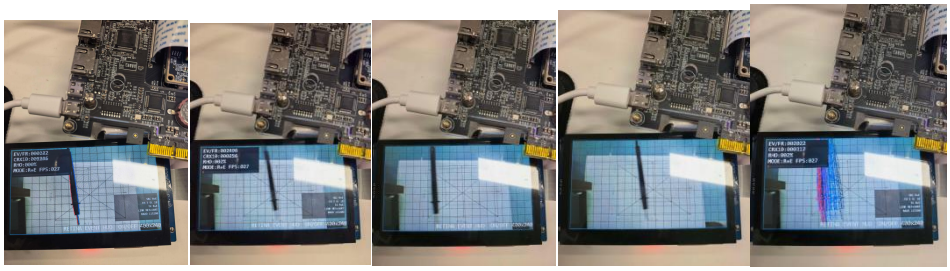


图 37　HUD 图层逐级关闭（a）关闭事件层（b）继续关闭至仅余网格

（4）　回传内容切换（丝印 KEY3）

按下后回传内容从“原画+事件”切到“纯事件”：上位机原画窗格停更、事件

窗格继续刷新，任务管理器里网口接收速率从约 164 Mbit/s 降到约 1 Mbit/s；再按一次恢复。纯事件模式是演示低带宽价值最直接的一步。

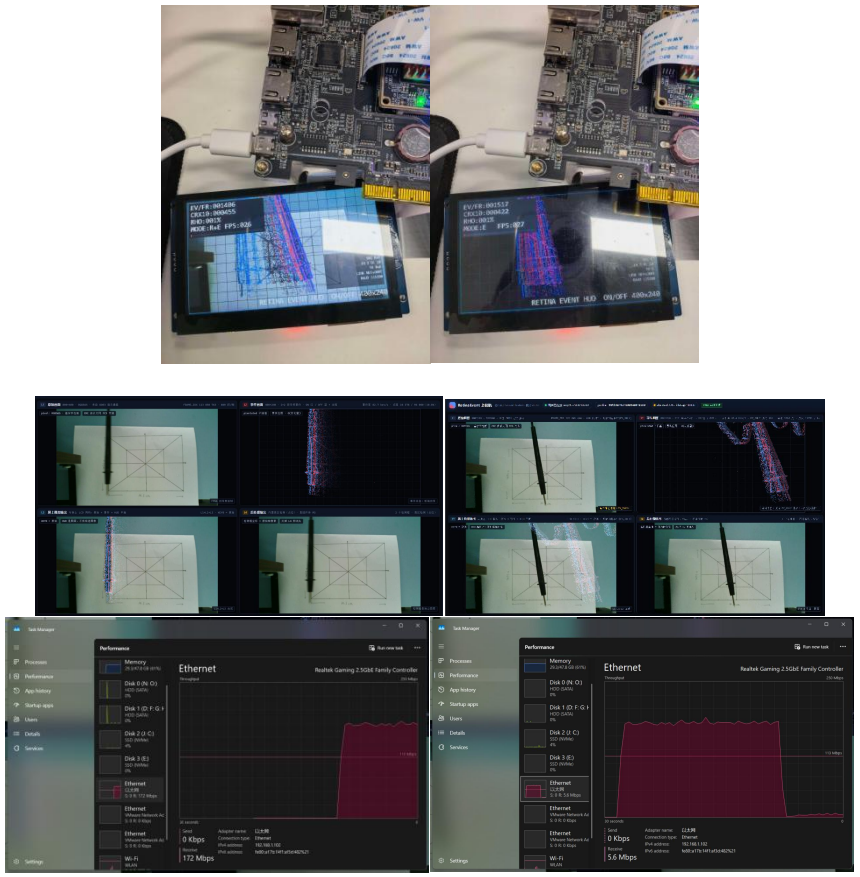


图 38　回传内容由「原画+事件」切到「纯事件」（a）板上 LCD（b）上位机原画窗格（c）网口带宽

第四部分　总结

4.1 可扩展之处

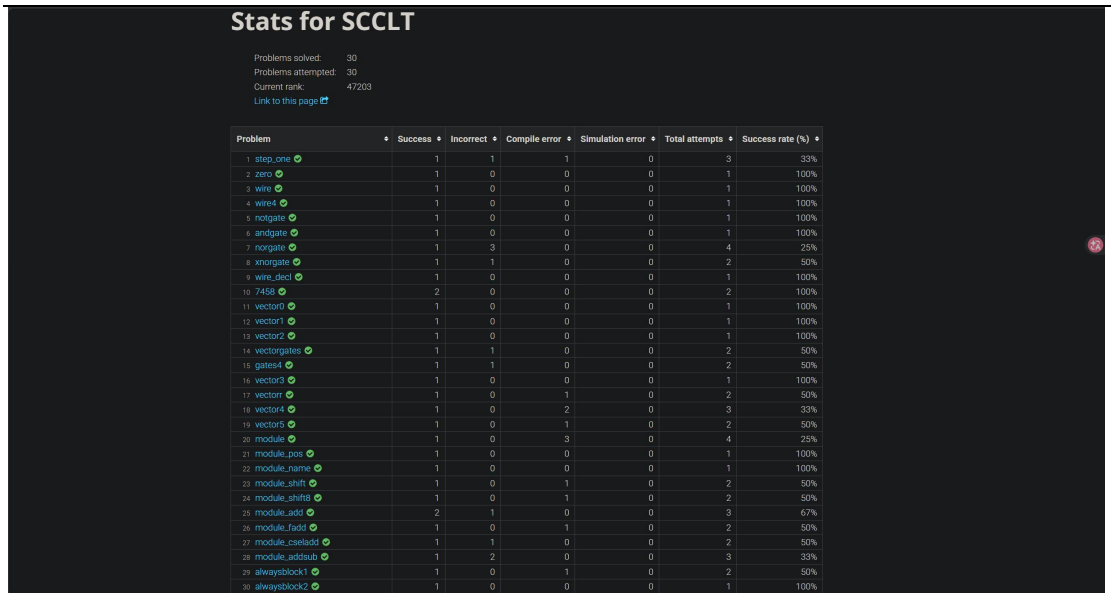
事件引擎目前只做一阶时域差分与空间中心-周边，再往上可以接多层时序滤波、运动方向编码与区域统计。神经网络侧现在只做二分类，算力余量足够扩到三到五类，这需要更多的数据采集与标注以提高模型泛化能力。回传侧可以可以借鉴视频编解码领域如 H.264 等压缩算法的思路，进一步压缩视频流和事件流所需的带宽，提高回传清晰度。板测可以自研 PCB 去除非必要部件并按需加大存储和接口能力，降本增效。

硬件上若换更大器件，可把 DDR3 也用于事件帧缓存，支持录像回看与断点续传。也可升级到 ZNYQ 架构的 FPGA 开发版，以获得板上系统的运行能力，同时可以尝试部署结构更复杂性能更强的模型，如循环门控，长短期记忆网络和 transfoemer 等，实现更通用的前端板载事件处理能力。

4.2 心得体会

作为初次参赛的二年级本科生，我们仅有单片机基础，没有 Verilog 开发经验，要完成本次项目作品是个不小的挑战。

自 9 月下旬决定参赛以来，我们几乎把所有的课余时间都投入到了比赛中。我们的学习从经典 Verilog 刷题网站开始，不到一周便刷完了 Verilog HDL 网站上的基础题；约半周后板卡到位，我们便以战代练，在开发中一边学习 FPGA 开发流程，一边深化 Verilog 编程能力。



Stats for SCCLT

Problems solved: 30
Problems attempted: 30
Current rank: 47203
Link to this page

	Problem	Success	Incorrect	Compile error	Simulation error	Total attempts	Success rate (%)
1	step_one	1	1	1	0	3	33%
2	zero	1	0	0	0	1	100%
3	wire	1	0	0	0	1	100%
4	wire4	1	0	0	0	1	100%
5	notgate	1	0	0	0	1	100%
6	andgate	1	0	0	0	1	100%
7	norgate	1	3	0	0	4	25%
8	xnorgate	1	1	0	0	2	50%
9	wire_decl	1	0	0	0	1	100%
10	7458	2	0	0	0	2	100%
11	vector0	1	0	0	0	1	100%
12	vector1	1	0	0	0	1	100%
13	vector2	1	0	0	0	1	100%
14	vectorgates	1	1	0	0	2	50%
15	gates4	1	1	0	0	2	50%
16	vector3	1	0	0	0	1	100%
17	vectorr	1	0	1	0	2	50%
18	vector4	1	0	2	0	3	33%
19	vector5	1	0	1	0	2	50%
20	module	1	0	3	0	4	25%
21	module_pos	1	0	0	0	1	100%
22	module_name	1	0	0	0	1	100%
23	module_shift	1	0	1	0	2	50%
24	module_shift8	1	0	1	0	2	50%
25	module_add	2	1	0	0	3	67%
26	module_fadd	1	0	1	0	2	50%
27	module_cseladd	1	1	0	0	2	50%
28	module_addsub	1	2	0	0	3	33%
29	alwaysblock1	1	0	1	0	2	50%
30	alwaysblock2	1	0	0	0	1	100%

此外，为在有限时间内完成作品开发，我们在调研中发现了一个可供 AI Agent 调用、以纯命令终端方式驱动 Vivado 的开源仓库 FPGA_Builder。结合为 AI 提炼的工作流 Skill，我们实现了 Verilog 编写—仿真—测试—综合—烧录—反馈的全闭环 FPGA 开发工作流，把常见于软件工程领域的敏捷开发模式引入 FPGA 开发，也让 LLM 的能力在开发中充分发挥，从而在有限时间内达到较高的作品完成度[8]。

经过长期开发与经验总结，我们形成了主对话中的多角色协作方式：由 leader 派活，指导 coder 和 reviewer 工作，并设置总监角色；不同角色的模型能力可以分别配置。例如，leader 使用 Opus、Astra、Kimi K3 等大尺寸模型，负责总体进度把关和计划设计；coder 使用 DS 的快速、低成本 Flash 模型，以大幅减少 Token 开销[10]。即使使用同一款模型（如 DS），多角色协作也能比单一 session 工作带来更高的完成度和完成质量：leader 只负责设计与把关，不接触底层技术细节，避免脏 Token 污染和上下文占用；同时，并行开发也大幅加速了开发进度。

当然，即使有 AI 辅助，人工仍是开发中不可替代的一环。AI 存在幻觉率，也受上下文长度限制，常忽略跨时钟域的时序细节。

最典型的一次是热插拔功能：早期每次插拔都会出现画面撕裂，人工复现、抓取波形并比对原始报文后，才定位到问题。我们发现根因不在显示链，而在帧号与像素的对齐。早先的打包逻辑用分片计数器回绕产生帧号，链路一旦丢失若

干像素或分片，计数就会永久错位，接收端把相邻两帧的像素拼在一起，画面便出现撕裂与错位。

解决办法是把帧边界做成随数据走带的标记：显示通路在每帧首像素产生 1 拍 marker，marker 与像素放在同一个 17 bit 跨域 FIFO 的同一个字中过域；打包器只在 marker 那一拍快照帧号，帧号不再由分片计数自增。配合复位后的首帧抑制和分片长度硬钳位，插拔与丢片后不再累积偏移。

经过本次比赛，我们的 FPGA 开发能力得到切实提升，也认识到 AI 在很大程度上是人类能力的放大器，而非“许愿机”。打铁还需自身硬，今后我们会把本次比赛总结的经验和教训应用到学习生活中，争取下次有更好的表现。

最后，衷心感谢 FPGA 创新设计赛道组委会与 AMD 提供本次参赛机会和开发平台，让我们得以在比赛中提升自己。

第五部分　参考文献

[1] AMD. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 创新设计赛道选题指南（AMD）[R]. 2026: 19-22.

[2] Lichtsteiner P, Posch C, Delbruck T. A 128×128 120 dB 15 μs latency asynchronous temporal contrast vision sensor[J]. IEEE Journal of Solid-State Circuits, 2008, 43(2): 566-576.

[3] Gallego G, Delbrück T, Orchard G, et al. Event-based vision: A survey[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 44(1): 154-180.

[4] AMD. Vivado Design Suite User Guide: Synthesis (UG901)[Z]. 2020.

[5] AMD. UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)[Z]. 2020.

[6] 正点原子. 达芬奇 Pro 开发板用户手册与原理图[Z]. 厂家资料.

[7] OmniVision. OV5640 CMOS 图像传感器数据手册[Z].

[8] FPGABuilder: 面向 FPGA 的仿真、综合、烧录一体化开源工具[EB/OL]. https://github.com/YiHok/FPGABuilder.

[9] 仿视网膜事件驱动视觉系统源码仓库[EB/OL]. https://github.com/p1gGod/RetinaEvent_FPGA.

[10] 黄思晓, 彭皓翔, 施旭, 等. 大模型 FPGA 推理实现技术综述与未来挑战[J]. 集成电路与嵌入式系统, 2025, 25(6): 1-13.